

Penetration Testing Frameworks

A penetration testing framework is a structured methodology that guides security professionals through every stage of an engagement, from initial planning and scoping to exploitation, reporting, and remediation validation.

1. [Open Source Security Testing Methodology Manual \(OSSTMM\)\(opens in new tab\)](#), a scientific, metrics-driven approach to security testing
2. [OWASP Web Security Testing Guide \(WSTG\)\(opens in new tab\)](#), the go-to framework for web application assessments
3. [NIST Special Publication 800-115\(opens in new tab\)](#), the U.S. government's technical guide to security testing and assessment
4. [Penetration Testing Execution Standard \(PTES\)\(opens in new tab\)](#), a practical, phase-driven standard that mirrors how real engagements are conducted
5. [Information Systems Security Assessment Framework \(ISSAF\)\(opens in new tab\)](#), a historically influential methodology with a detailed nine-step assessment model

also introduce [MITRE ATT&CK\(opens in new tab\)](#) as a complementary knowledge base that maps adversary tactics and techniques. We will survey several other notable frameworks, including the [WASC Threat Classification\(opens in new tab\)](#), [CSA Cloud Controls Matrix\(opens in new tab\)](#), [OWASP Mobile Application Security Testing Guide \(MASTG\)\(opens in new tab\)](#), [PCI DSS Penetration Testing Guidelines\(opens in new tab\)](#), and the [CBEST Framework\(opens in new tab\)](#), so you know when and where they apply.

OSSTMM

Open Source Security Testing Methodology Manual

OSSTMM organises testing around five **security channels**, reflecting its philosophy that security is not just a network problem:

- **Human Security (HUMSEC)**: Social engineering and human-factor vulnerabilities.
- **Physical Security (PHYSSEC)**: Physical access controls, from badge readers to tailgating.
- **Wireless Communications (SPECSEC)**: Wi-Fi, Bluetooth, RFID, and other electromagnetic signals.
- **Telecommunications (COMSEC)**: Phone systems, VoIP, fax, and modem infrastructure.
- **Data Networks (DATASEC)**: Network services, firewalls, and application-layer protocols.

Phases: A Walkthrough

Phase 1: Induction covers enumeration and verification. You map what exists and confirm it is real. At FinVault, you query DNS, review certificate transparency logs, and discover subdomains like `vpn.finvault-corp.thm` and `mail.finvault-corp.thm`. You then verify each asset is live and responsive. The output is a confirmed inventory of the target environment.

Phase 2: Interaction covers qualification and quantification. You actively probe the verified assets and assess their relevance. At FinVault, you connect to each service, fingerprint its technology, and quantify the exposure: 12 externally reachable services across 8 hosts, 4 of which accept unauthenticated connections. These findings feed directly into the attack surface calculation.

Phase 3: Inquiry covers privilege escalation and verification escalation. You test whether the measured exposure can be converted into unauthorized access. At FinVault, you discover an IDOR vulnerability in the customer portal that allows one authenticated user to read another customer's account statements. Verification escalation confirms the scope: read access to 12,000 accounts, no write access.

Phase 4: Intervention covers quarantine, audit, and enticement. You address findings and examine the broader control environment. At FinVault, the vulnerable endpoint is restricted. At the same time, a patch is developed (quarantine), the wider access control model is examined for similar flaws (audit), and a canary token is deployed to test internal detection capabilities (enticement).

OSSTMM's quantitative approach are **Risk Assessment Values (RAVs)**

Closing Notes

OSSTMM prescribes the **Security Test Audit Report (STAR)** format for deliverables, enforcing consistency and enabling cross-team comparability.

OSSTMM is best suited for organizations that need repeatable, auditable security measurements and are willing to invest in mastering the methodology.

OWASP WSTG

While OWASP is most widely known for the [OWASP Top Ten \(opens in new tab\)](#) list of critical web vulnerabilities, the WSTG goes far deeper: it is a comprehensive, community-driven framework that organizes web application testing into **over 90 discrete test cases** grouped across twelve categories.

include information gathering, configuration and deployment management, identity management, authentication, authorization, session management, input validation, error handling, cryptography, business logic, client-side testing, and API testing. Each category

contains numbered test cases (e.g., WSTG-INPV-01 for reflected cross-site scripting) with step-by-step guidance on what to test and how to test it.

The WSTG adopts a **risk-based approach**: vulnerabilities are prioritized based on their exploitability and impact, not simply cataloged.

Phases: Security Across the SDLC

aligns testing across five phases of the **Software Development Life Cycle (SDLC)**

Phase 1: Before development begins. Security requirements and regulatory obligations are established upfront. For ShopSecure, this means defining that the portal must comply with **PCI DSS** (since it processes payments) and establishing measurable criteria, such as the maximum acceptable time for patching vulnerabilities.

Phase 2: During definition and design. The application architecture is reviewed for security flaws before any code is written. ShopSecure's team creates threat models for the payment flow, identifying that the checkout API will be a high-value target and designing rate-limiting and input validation controls from the start.

Phase 3: During development. Code is vetted through walkthroughs and reviews. ShopSecure's developers review the authentication module against WSTG test cases for credential handling (WSTG-ATHN), identifying a flaw in which password reset tokens do not expire.

Phase 4: During deployment. Security controls are verified in the production environment. ShopSecure's team runs a penetration test against the staged application, verifying that default credentials have been changed, that TLS is properly configured, and that no debug endpoints are exposed.

Phase 5: During maintenance and operations. Security is maintained post-launch through periodic health checks, especially after updates. When ShopSecure pushes a new product recommendation feature three months later, the relevant WSTG test cases are re-executed to ensure the update has not introduced new vulnerabilities.

Closing Notes

The WSTG's greatest strength is its **exhaustive, practical coverage**. With over 90 test cases, each containing specific procedures and expected results, it gives testers a concrete roadmap rather than abstract principles.

a **full implementation of every test case can be impractical** for resource-constrained teams. Some tests require specialized expertise in areas like cryptographic analysis or business logic testing.

NIST SP 800-115

this document provides a foundational framework for systematically evaluating the security posture of information systems.

NIST SP 800-115 is not a penetration testing framework in the narrow sense. It is broader: it covers the full spectrum of **security testing and assessment techniques**, from document reviews and log analysis to vulnerability scanning and full penetration tests.

Three core objectives drive the framework. First, **identify vulnerabilities** in systems, networks, and applications. Second, **validate security controls** by testing whether they perform as expected under adversarial conditions. Third, **assess exploitability** by simulating real-world attack scenarios to determine whether a threat actor can actually leverage identified weaknesses.

Phases: A Walkthrough

Phase 1: Planning. Before any testing begins, the objectives, scope, and rules of engagement are formally defined and documented. For GovNet, this means specifying which network segments are in scope (the citizen portal and its supporting backend) and which are excluded (the classified enclave). It also means establishing communication protocols: who gets notified if a critical vulnerability is found mid-test, what hours testing is permitted, and what constitutes an emergency stop condition. The planning phase produces a formal test plan that all stakeholders sign off on.

Phase 2: Execution. This is where active testing happens. NIST SP 800-115 groups execution activities into four technique categories:

- **Review techniques:** Examining documentation, policies, system configurations, and rule sets. At GovNet, this includes reviewing firewall rules and access control lists for misconfigurations.
- **Target identification and analysis:** Discovering and fingerprinting live hosts, open ports, and running services. At GovNet, you scan the citizen portal's infrastructure and identify 12 internet-facing services.
- **Target vulnerability validation:** Confirming that identified weaknesses are real and exploitable, not false positives. At GovNet, a scan flags a potential SQL injection on the portal's search function; you manually validate it by crafting a test query that returns database version information.
- **Penetration testing:** Simulating adversarial attacks to test the depth of exploitation possible. At GovNet, you chain the confirmed SQL injection with a privilege escalation on the database server to demonstrate that an external attacker could access internal citizen records.

Notice how NIST SP 800-115 treats these as a progression from passive to active, from low-impact to high-impact. Not every engagement needs to reach the penetration testing stage; sometimes, a review and vulnerability validation are sufficient for the assessment's objectives.

Phase 3: Post-Testing. The focus shifts to analyzing results, prioritizing risks, and delivering actionable remediation strategies. For GovNet, this means categorizing findings by severity (the SQL injection chain is critical; a missing HTTP security header is low), mapping each finding to the specific control it bypasses, and providing concrete remediation steps. NIST SP 800-115 emphasizes that findings should be **actionable**: telling a client "you have a SQL injection" is not enough; explaining which parameter is vulnerable, what data is at risk, and how to remediate it is the standard.

Closing Notes

NIST SP 800-115's strengths lie in its **flexibility and institutional credibility**. It adapts to diverse environments, from traditional data centers to cloud infrastructure and IoT deployments, because it defines technique categories rather than prescribing specific tools.

PTES

Available at pentest-standard.org(opens in new tab), PTES was developed by a group of experienced security practitioners with a specific goal: to define what a real penetration test looks like, end-to-end. Where other frameworks focus on *what* to test or *how* to measure, PTES focuses on *how the engagement flows* from start to finish.

PTES is organized into **seven phases** that map directly to the lifecycle of a penetration testing engagement. This approach makes it exceptionally practical for junior testers because it answers the question that many frameworks leave implicit: "I have a signed contract; now what do I do on day one, day two, and every day after?"

Phases: A Walkthrough

Phase 1: Pre-Engagement Interactions. This is everything that happens before testing begins. You define the scope with MedGuard's IT director: the corporate LAN (10.10.0.0/16), the patient portal at `records.medguard-health.thm` , and wireless networks at the headquarters building. You document the rules of engagement, including testing windows (weeknights only to avoid disrupting clinical operations), emergency contacts, and a "get out of jail free" letter authorizing the test. PTES is notably detailed here because unclear scoping is the number one source of legal and professional disputes in penetration testing.

Phase 2: Intelligence Gathering. You collect information about MedGuard using both passive and active techniques. Passive reconnaissance includes harvesting employee email addresses from LinkedIn, discovering subdomains through certificate transparency logs, and reviewing job

postings that reveal technology stacks ("seeking a DBA with Oracle 19c experience"). Active reconnaissance involves DNS enumeration and network scanning within the agreed scope. PTES distinguishes between these levels because the depth of intelligence gathering directly shapes the quality of the subsequent phases.

Phase 3: Threat Modeling. Using the intelligence gathered, you identify the most valuable targets and the most likely attack paths. At MedGuard, the patient records database is the highest-value asset. Your threat model identifies two primary attack paths: compromising the patient portal directly through a web vulnerability, or pivoting through the corporate LAN after compromising an employee workstation. This phase ensures your testing effort is directed by adversarial logic rather than random scanning.

Phase 4: Vulnerability Analysis. You systematically identify weaknesses that could enable the attack paths from your threat model. At MedGuard, vulnerability scanning reveals that the patient portal is running an outdated version of Apache Tomcat, which contains a known deserialization vulnerability. On the internal network, several workstations are missing critical patches. PTES emphasizes that vulnerability analysis includes both automated scanning and manual verification to eliminate false positives.

Phase 5: Exploitation. You attempt to exploit the confirmed vulnerabilities. At MedGuard, you exploit the Tomcat deserialization flaw to gain a shell on the portal server. On the internal side, you use a phishing pretext (authorized in the scope) to deliver a payload to an employee workstation. PTES stresses that exploitation should be purposeful: the goal is to demonstrate business impact, not to "pop boxes" for the sake of it.

Phase 6: Post-Exploitation. After gaining access, you determine the real-world impact. From the compromised portal server, you pivot into the backend database and confirm read access to patient records. From the employee workstation, you extract cached domain credentials and demonstrate lateral movement to a file server containing financial data. PTES treats post-exploitation as the phase where technical findings are translated into business risk: "we accessed 50,000 patient records" carries far more weight than "we got a shell."

Phase 7: Reporting. You deliver the findings in a structured report with two audiences in mind. The **executive summary** communicates business risk in plain language for MedGuard's leadership: patient data was accessible, regulatory exposure under HIPAA is significant, and remediation is urgent. The **technical report** provides the details that MedGuard's IT team needs to reproduce and fix each finding: exact exploitation steps, affected hosts, evidence screenshots, and prioritized remediation guidance.

Closing Notes

PTES's greatest strength is its **practical, end-to-end structure**. It reads like a playbook for how engagements actually unfold, which makes it an excellent learning framework for junior

testers building their workflow instincts.

PTES has **not been formally updated in several years**, and the technical guidance sections reference outdated tools and techniques. The methodology and phase structure remain sound, but testers should supplement PTES's tool-specific guidance with current documentation.

ISSAF

Information Systems Security Assessment Framework (ISSAF)

an open-source penetration testing framework designed to evaluate network, system, and application security.

ISSAF's **nine-step assessment model** is one of the clearest representations of how an attacker progresses through a target environment.

Phases: A Walkthrough

Phase 1: Planning and Preparation

This phase sets the engagement boundaries. You meet with TechBridge's CTO to define the scope (corporate network, Git server, and the project management portal), establish escalation protocols and emergency contacts, identify constraints (the production Git server must not be disrupted during business hours), and agree on the toolset appropriate for the assessment.

Phase 2: Assessment

This is the core of ISSAF and where its nine-step model lives. Each step builds on the previous one, simulating how a real adversary would progress through the environment.

1. **Information gathering:** Collect publicly available data about TechBridge. DNS records, WHOIS data, employee profiles on LinkedIn, and technology references in job postings ("experience with Jenkins and GitLab CI required") all feed your understanding of the target.
2. **Network mapping:** Map the live network topology. You discover TechBridge's external IP range hosts the project portal, a VPN gateway, and a mail server. Internal scanning (once in scope) reveals the Git server, a Jenkins build server, and several developer workstations.
3. **Vulnerability identification:** Scan the mapped assets for weaknesses. The project portal runs an outdated CMS with a known authentication bypass. The Jenkins server has its administrative console exposed without authentication.
4. **Penetration:** Attempt initial exploitation. You exploit the unauthenticated Jenkins console to execute system commands on the build server.
5. **Gaining access and privilege escalation:** Escalate from initial access to higher privileges. From the Jenkins server, you recover stored credentials for the service account that deploys code to production, which has administrative rights on the Git server.

6. **Enumerating further:** With elevated access, enumerate what is now reachable. From the Git server, you discover repositories containing API keys, database connection strings, and client project source code.
7. **Compromise remote users/sites (lateral movement):** Move laterally to other systems. Using the harvested credentials, you access several developer workstations and the internal mail server.
8. **Maintaining access:** Establish persistent access to demonstrate that a real attacker could retain their foothold. You document (without actually deploying) how a backdoor could be planted in the CI/CD pipeline, persisting across system reboots and deployments.
9. **Covering tracks:** Demonstrate how an attacker would erase evidence. You document which logs captured your activity and identify gaps in TechBridge's logging that would allow a real adversary to operate undetected.

Phase 3: Reporting and Cleanup

You compile findings into a structured report, prioritized by business impact. The unauthenticated Jenkins console is flagged as critical because it provided the initial foothold that led to source code access. Cleanup involves removing any test artifacts, revoking any temporary accounts created during testing, and confirming with TechBridge's team that no testing residue remains in their environment.

Closing Notes

The progression from information gathering through lateral movement to covering tracks provides a clear mental model for how real-world attacks unfold, making it an excellent educational tool even though the framework itself is no longer maintained.

ISSAF should be studied for its **methodology and adversarial logic**, not for its technical procedures.

MITRE ATT&CK

ATT&CK stands for **Adversarial Tactics, Techniques, and Common Knowledge**. Developed and maintained by [MITRE Corporation](#)(opens in new tab), it is not a traditional penetration testing framework. It is a **knowledge base** of adversary behavior, built from real-world observations of how threat actors operate. It catalogs *what* attackers do, organized in a structure that security professionals can use for threat intelligence, detection engineering, red teaming, and, relevant to this room, enriching penetration test findings.

The Matrix: Tactics, Techniques, and Sub-Techniques

The current Enterprise matrix includes 14 tactics, progressing from initial access through execution, persistence, privilege escalation, defense evasion, credential access, discovery,

lateral movement, collection, command and control, exfiltration, and impact.

under the Initial Access tactic, you will find techniques like Phishing (T1566), Exploit Public-Facing Application (T1190), and Valid Accounts (T1078). Many techniques are further broken down into **sub-techniques**.

ATT&CK as a Complement, Not a Replacement

provides a **common language** for describing what you found during a test conducted using any framework.

Walkthrough: Mapping Findings to ATT&CK

Engagement Finding	ATT&CK Tactic	ATT&CK Technique
Phishing email delivered payload to employee workstation	Initial Access	Phishing: Spearphishing Attachment (T1566.001)
Exploited Tomcat deserialization flaw on patient portal	Initial Access	Exploit Public-Facing Application (T1190)
Extracted cached domain credentials from workstation	Credential Access	OS Credential Dumping (T1003)
Moved from workstation to file server using stolen credentials	Lateral Movement	Use Alternate Authentication Material (T1550)
Accessed patient records database from compromised portal server	Collection	Data from Information Repositories (T1213)

Closing Notes

enables penetration testers, threat intelligence analysts, detection engineers, and incident responders to speak the same language.

Other Notable frameworks

Framework	Primary Domain	Type	Actively Maintained	When You Would Use It
WASC Threat Classification	Web applications	Threat taxonomy	No (superseded by OWASP)	Legacy references; historical context
CSA Cloud Control Matrix	Cloud environments	Governance/compliance controls	Yes	Cloud security posture assessments
OWASP Mobile Application Security Testing Guide	Mobile applications (Android/iOS)	Testing guide	Yes	Mobile app penetration testing
PCI DSS Penetration Testing Guidelines	Payment card environments	Regulatory mandate	Yes (current: PCI DSS v4.0)	Any engagement involving cardholder data
CBEST	UK financial sector	Threat-intel-led pen testing	Yes	Engagements with UK financial institutions

Choosing the Right Framework

Selection Criteria

Four factors consistently drive framework selection in real-world engagements.

Engagement scope and target type is the most immediate filter. A web application assessment aligns with the OWASP WSTG. A mobile app engagement calls for OWASP MASTG. A full-spectrum network penetration test aligns naturally with PTES or OSSTMM. If the scope spans multiple channels, including physical and human factors, OSSTMM's five-channel model becomes particularly relevant.

Regulatory and compliance requirements can override personal preference entirely. If the client processes payment card data, PCI DSS penetration testing guidelines are not optional. If the client is a UK financial institution subject to Bank of England oversight, CBEST may be mandated. NIST SP 800-115 carries weight in U.S. government and federal contractor environments. When regulation dictates, the tester adapts.

Need for quantifiable, repeatable results matters when multiple assessors are involved or when results must be compared across time periods. OSSTMM's RAV metrics are specifically designed for this. If a client says, "We want to measure whether our security posture has improved since last year's test," OSSTMM provides the measurement framework to objectively answer that question.

Team expertise and available resources are practical constraints that are easy to overlook. A full OSSTMM implementation requires deep familiarity with its metrics system. CBEST demands threat intelligence capabilities. For a smaller team conducting a standard corporate network assessment, PTES offers a practical, well-structured workflow without the overhead of more specialized frameworks.

Scenario Practice

Scenario 1:

A regional hospital hires your team to test their patient-facing web portal and the internal network it connects to. The hospital must comply with HIPAA, but is not subject to PCI DSS (it outsources payment processing). They want a structured, end-to-end assessment with a clear executive summary for the board of directors.

PTES is the strongest fit here. It provides the end-to-end engagement structure the hospital needs, its reporting phase explicitly produces both executive and technical deliverables, and it covers both web application and network testing within a single methodology. OWASP WSTG can supplement the web portal testing specifically, and mapping findings to MITRE ATT&CK technique IDs would strengthen the report by grounding it in real-world threat behavior.

Scenario 2:

A multinational bank headquartered in London needs a penetration test that satisfies its UK regulatory obligations. The engagement must begin with a threat intelligence assessment specific to the institution.

CBEST is the clear choice. It is designed for UK financial institutions, mandates a bespoke threat intelligence phase, and is recognized by the Bank of England. No general-purpose framework alone would satisfy these requirements.

Scenario 3:

A SaaS startup wants two different security firms to independently test its platform over the next two years and compare the results directly to track improvement. The platform is entirely web-based.

OSSTMM's RAV metrics enable direct cross-team, cross-time comparisons that the client is requesting. OWASP WSTG would guide the web-specific test cases, while OSSTMM provides the quantitative measurement layer that makes year-over-year comparison meaningful.

Scenario 4:

A fintech company asks you to test their Android and iOS mobile banking applications, both of which handle credit card transactions.

This engagement requires two frameworks working together. OWASP MASTG provides the test cases for mobile application security on both platforms. PCI DSS penetration testing guidelines must also be followed because the applications handle cardholder data. The tester needs to satisfy both the mobile-specific testing methodology and the regulatory mandate simultaneously.

Conclusion

Master Comparison Table

Framework	Scope	Key Strength	Limitation	Best For
OSSTMM	Multi-channel (5 channels)	Quantifiable metrics (RAVs)	Steep learning curve	Repeatable, auditable assessments
OWASP WSTG	Web applications	90+ structured test cases	Can encourage checklist mentality	Web app penetration testing
NIST SP 800-115	Systems, networks, applications	Federal credibility; flexible technique categories	Guidance only, no enforcement	Government and enterprise assessments
PTES	End-to-end engagements	Practical, phase-driven workflow	Not recently updated	Standard network/application pentests
ISSAF	Networks, systems, applications	Clear nine-step adversarial progression	No longer maintained	Educational reference for attack methodology
MITRE ATT&CK	Adversary behavior catalog	Universal language for threat behavior	Not a testing methodology	Enriching findings from any framework
OWASP MSTG	Mobile applications (Android/iOS)	Platform-specific mobile test cases	Mobile-only scope	Mobile app security testing

PCI DSS Guidelines	Payment card environments	Regulatory mandate	Narrow applicability	Engagements involving cardholder data
CSA CCM	Cloud environments	Governance controls mapped to major standards	Not a penetration testing methodology	Cloud security posture assessments
CBEST	UK financial sector	Threat-intel-led, regulatory backing	UK financial sector only	UK financial institution engagements
WASC Threat Classification	Web applications	Historical taxonomy	Superseded by OWASP	Legacy reference only